

E-Commerce Inventory System

Project Report

Course: CSE3001 - Database Management Systems

Institution: VIT Bhopal

Database: MySQL 8.x (ecommerce_inventory)

Team Members

Tanuj Tanmay Patel	25bce11280
Hemil Shah	25bce10124
Prashant Taparia	25bce11043
Arun Kumar	25bce11009

1. Abstract

This project designs and implements a relational database for the inventory side of a single-location online store. It tracks product categories, products with stock and reorder levels, suppliers and their supply costs, customers, and the orders they place. Business rules are enforced inside the database itself: constraints keep prices and quantities valid, a trigger reduces stock when an order line is added, and another trigger rejects orders that exceed available stock. Two views provide ready-made reports for low-stock alerts and order totals.

2. Problem Statement

Small online stores often track stock in spreadsheets, which leads to overselling, duplicated data, and no reliable record of who supplies what at what cost. The goal is a normalized database that:

- keeps one consistent record of products, suppliers, customers and orders,
- prevents invalid data (negative prices, zero quantities, orders beyond stock),
- updates stock automatically as orders are placed,
- answers common business questions with simple SQL.

3. Objectives

1. Model the domain as an ER diagram and convert it to a relational schema.
2. Normalize all tables to 3NF/BCNF.
3. Enforce integrity with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and ENUM.
4. Automate stock management with triggers.
5. Provide reporting views and example analytical queries.

4. Scope and Assumptions

- Single store location, so one stock figure per product.
- Each product belongs to exactly one category.
- A product can have several suppliers, and a supplier can supply several products.
- An order belongs to one customer and holds one line per distinct product.
- Prices are in Indian Rupees (INR).

5. ER Model

5.1 Entities

Entity	Purpose
Category	Groups products
Product	Items sold, with price, stock and reorder level
Supplier	Vendors who supply products
Product_Supplier	Links products to suppliers, with cost price and supply date
Customer	People who buy
Orders	One purchase by a customer, with date and status
Order_Item	Products inside an order, with quantity and unit price

5.2 Relationships

Relationship	Cardinality	Implementation
Category has Product	1:N	FK Product.category_id
Product supplied by Supplier	N:N	Junction table Product_Supplier (PK: product_id, supplier_id)
Customer places Orders	1:N	FK Orders.customer_id
Orders has Order_Item	1:N	FK Order_Item.order_id
Product contains Order_Item	1:N	FK Order_Item.product_id

Orders and Products are many-to-many (an order holds many products, and a product appears in many orders). This is resolved by **Order_Item**, whose primary key is (order_id, product_id).

Note on "has": "has" is only the verb written in the relationship diamond, not a table. 1:N relationships become a foreign key, and only N:N relationships become a separate table.

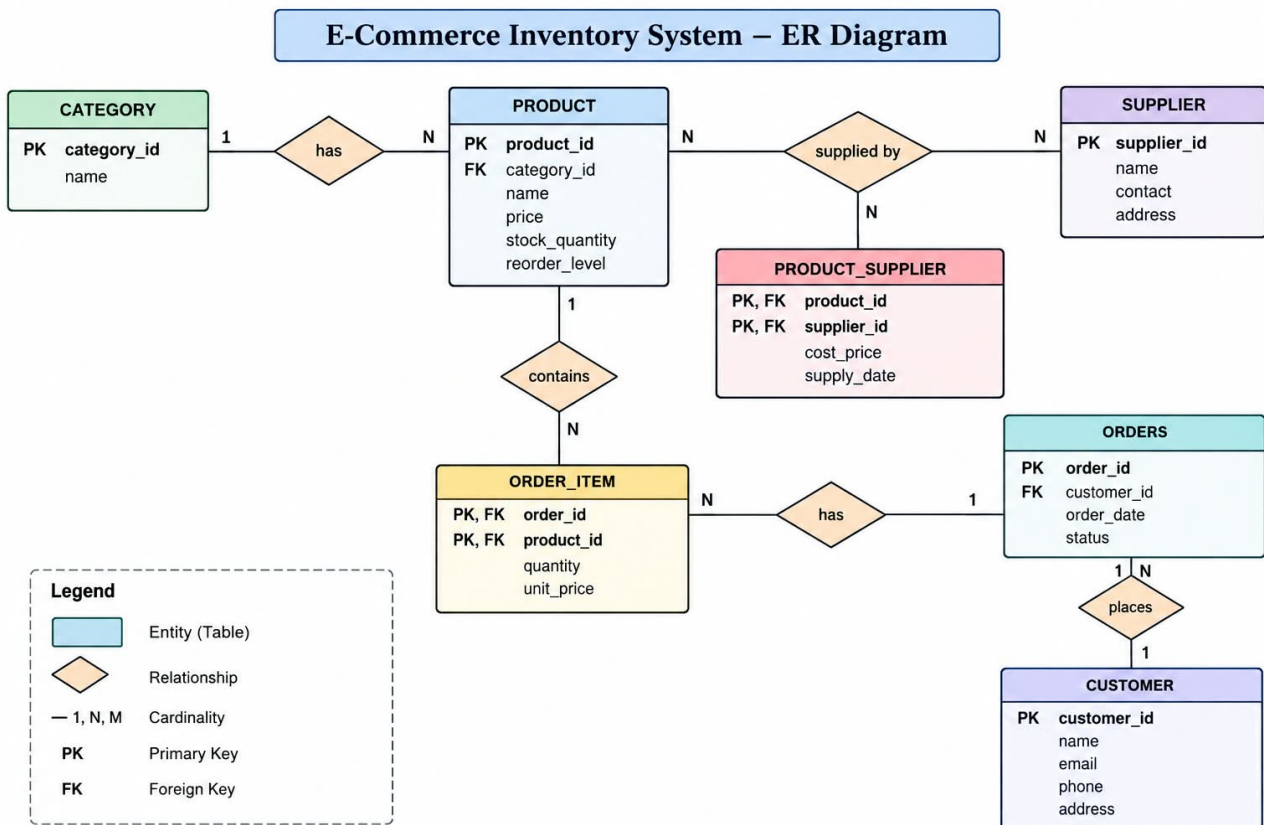


Figure 1: ER diagram of the E-Commerce Inventory System

6. Relational Schema

Table	Columns (primary keys underlined)
Category	<u>category_id</u> , name (unique)
Product	<u>product_id</u> , name, category_id (FK), price, stock_quantity, reorder_level
Supplier	<u>supplier_id</u> , name, contact, address
Product_Supplier	<u>product_id</u> (FK), <u>supplier_id</u> (FK), cost_price, supply_date
Customer	<u>customer_id</u> , name, email (unique), phone, address
Orders	<u>order_id</u> , customer_id (FK), order_date, status
Order_Item	<u>order_id</u> (FK), <u>product_id</u> (FK), quantity, unit_price

Primary key columns are underlined.

6.1 Constraints

Constraint	Where	Purpose
CHECK (price >= 0)	Product	No negative prices
CHECK (stock_quantity >= 0)	Product	Stock cannot go negative
CHECK (cost_price >= 0)	Product_Supplier	Valid supplier cost
CHECK (quantity > 0)	Order_Item	Each line has at least one unit
CHECK (unit_price >= 0)	Order_Item	Valid line price
UNIQUE	Category.name, Customer.email	No duplicate categories or accounts
ENUM (Pending, Shipped, Delivered, Cancelled)	Orders.status	Only valid order states
DEFAULT	Product, Orders	Stock 0, reorder level 10, status Pending, date = today

6.2 Referential Actions

Foreign key	ON DELETE	ON UPDATE	Reason
Product.category_id	RESTRICT	CASCADE	Cannot delete a category that still has products
Product_Supplier (both FKs)	CASCADE	CASCADE	Links disappear with their product or supplier
Orders.customer_id	RESTRICT	CASCADE	Cannot delete a customer who has orders
Order_Item.order_id	CASCADE	CASCADE	Deleting an order removes its lines
Order_Item.product_id	RESTRICT	CASCADE	Cannot delete a product that has been sold

6.3 Indexes

Index	Column	Speeds up
idx_product_category	Product(category_id)	Category-wise product lookups
idx_order_customer	Orders(customer_id)	A customer's order history
idx_orderitem_product	Order_Item(product_id)	Sales per product

7. Normalization

- **1NF:** every column holds a single atomic value, and every table has a primary key.
- **2NF:** in the junction tables, non-key attributes depend on the whole composite key. cost_price and supply_date depend on the (product, supplier) pair, and quantity and unit_price depend on the (order, product) pair.
- **3NF/BCNF:** there are no transitive dependencies. Category names live only in Category, customer details only in Customer, and every determinant is a candidate key.

Order_Item.unit_price deliberately copies the product price at the time of purchase. It records history, so a later change to Product.price does not alter past orders. This is a snapshot of a fact about the order line, not redundancy.

8. Implementation

8.1 Files

File	Contents
schema.sql	Database creation, tables, constraints, indexes
sample_data.sql	Sample rows for all tables
triggers_and_views.sql	Two triggers and two views
queries.sql	Eight example queries

8.2 Setup

```
mysql -u root -p < schema.sql
mysql -u root -p < sample_data.sql
mysql -u root -p < triggers_and_views.sql
mysql -u root -p ecommerce_inventory < queries.sql
```

8.3 Triggers

Trigger	Event	Behaviour
trg_check_stock_before_order	BEFORE INSERT on Order_Item	Reads current stock; if it is less than the requested quantity, raises SQLSTATE 45000 with "Insufficient stock for this product" and the insert is rejected
trg_reduce_stock_after_order	AFTER INSERT on Order_Item	Subtracts the ordered quantity from Product.stock_quantity

Together they keep stock accurate and prevent overselling.

8.4 Views

View	Description
vw_low_stock_products	Products whose stock is at or below reorder_level, with category name, used as a reorder alert list
vw_order_summary	One row per order with customer name, date, status and order_total (sum of quantity x unit_price)

9. Sample Data

Table	Rows	Notes
Category	5	Electronics, Apparel, Home & Kitchen, Books, Sports
Product	8	e.g. Wireless Mouse (Rs. 799), Mechanical Keyboard (Rs. 2999), Cricket Bat (Rs. 1499)
Supplier	3	TechDistributors, FashionHub Wholesale, HomeEssentials Co.
Product_Supplier	6	Six products are linked to one supplier each
Customer	3	Aarav Sharma, Priya Nair, Rohan Mehta
Orders	3	Delivered, Shipped, Pending
Order_Item	4	Four order lines across the three orders

10. Queries and Expected Results

#	Purpose	Technique	Result on sample data
1	Products with category name	JOIN	All 8 products with their category
2	Products with more than one supplier	JOIN, GROUP BY, HAVING	No rows (each product has one supplier)
3	Total spent per customer	Multi-table JOIN, SUM, ORDER BY	Aarav Sharma Rs. 4,047; Priya Nair Rs. 899
4	Products never ordered	Subquery with NOT IN	Mechanical Keyboard, Denim Jacket, Blender, Cricket Bat
5	Best-selling product by quantity	SUM, ORDER BY, LIMIT 1	Database Systems Book (3 units)
6	Order summaries	View	Order 1 Rs. 2,097; Order 2 Rs. 899; Order 3 Rs. 1,950
7	Low-stock list	View	No rows (all stock above reorder levels)
8	Suppliers of Electronics products	Multi-table JOIN, DISTINCT	TechDistributors Pvt Ltd

Queries 2 and 7 return no rows because the sample data is small. Adding a second supplier for a product, or lowering a product's stock, produces results.

11. Testing

Test	Action	Expected outcome
Stock reduction	Insert an Order_Item of quantity 2 for a product with stock 50	Stock becomes 48
Overselling	Insert an Order_Item whose quantity exceeds stock	Error "Insufficient stock for this product"; no row inserted
Invalid price	Insert a Product with price -1	CHECK violation
Zero quantity	Insert an Order_Item with quantity 0	CHECK violation
Duplicate email	Insert a second Customer with an existing email	UNIQUE violation
Protected delete	Delete a Category that has products	Blocked by RESTRICT
Cascade delete	Delete an Order	Its Order_Item rows are removed
Low-stock alert	Reduce a product's stock to its reorder level	Product appears in vw_low_stock_products

12. Limitations and Future Scope

Limitations

- Sample orders are inserted before the triggers exist (the setup order in 8.2), so their quantities are not subtracted from stock. Stock reduction applies only to orders added afterwards.
- Cancelling an order or deleting an order line does not return stock, because there is no UPDATE or DELETE trigger.
- Changing the quantity of an existing order line is not checked against stock.
- Only one stock figure exists per product, so multiple warehouses are not supported.

Future scope

- Add AFTER UPDATE (status changed to Cancelled) and AFTER DELETE triggers to restore stock.
- Add a purchase-order table so supplier deliveries increase stock automatically.
- Add payments, shipping addresses and user roles.
- Add stored procedures for placing an order as a single transaction.
- Add a front-end application connected to the database.

13. Conclusion

The project delivers a seven-table normalized schema that matches its ER diagram, with integrity rules enforced in the database, automatic stock control through triggers, and views and queries for everyday reporting. The design is simple to extend toward a fuller inventory and order-management system.